AI INFRA RESEARCH SERIES

HiSparse

稀疏注意力省了计算，为什么还会被显存卡住?

分析对象

HiSparse: Scaling Sparse-Attention Decoding with Hierarchical KV Cache Management

作者与机构

Zhiqiang Xie、Zhangheng Huang、Tingwei Huang、Ziyi Xu、Ruiyang Ma、Christos Kozyrakis；Stanford/Meta、阿里巴巴、蚂蚁集团、上海交大、北京大学、NVIDIA Research。

版本范围

arXiv:2608.07009v1，2026-08-07；重点核对方法、CUDA Graph 内缓存管理及作者报告的 SGLang 实验。

核心判断

HiSparse 不重新决定哪些 token 重要，而是利用已有稀疏选择器给出的精确访问集合，把完整 KV 放在主机内存，只在 GPU 上保留有限热点工作集。收益主要来自解除并发容量限制，而不是让每次注意力计算凭空更快。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 一句话定位与最容易混淆的边界 2

2 容量墙：少读并不等于少存 2

3 架构与一个请求的完整生命周期 3

4 CUDA Graph 里的动态缓存管理 3

5 跨层预取：可证明的复用比猜测更有价值 4

6 实验结果：收益主要来自更大的可服务并发 4

7 缓存命中率实验究竟说明什么 5

8 一个可用于复现的成本模型 6

9 exact 语义的验收要比性能更先做 6

10 局限、部署代价与研究空间 7

11 复现顺序与资料 7

文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 一句话定位与最容易混淆的边界

稀疏 attention 每次只读少量 KV，但下一次可能选择历史中完全不同的位置。因此，很多系统仍把完整历史放在 HBM：**访问稀疏，驻留却不稀疏**。HiSparse 正是解决这个不匹配。[S1，§1]

它接受 DSA、NSA、Quest 等选择器产生的逻辑位置，保证这些位置对应的 KV 在 attention 开始前出现在 GPU 的正确物理槽位。选择器和 attention 数学不需要因为换了存储层而改变。[S1，§2–3]

技术	决定什么	本身是否改变 attention 的选择
HiLS	选择哪些块、各块如何加权	是，属于模型机制
HiSparse	所选 KV 存在哪里、何时取回	不改变已有选择
KDA	用递归矩阵状态怎样承载历史	是，改变历史表示与读取
KV 量化	每个 KV 元素如何编码	可能引入数值近似

这里的 exact 是**相对所采用的稀疏模型／选择器**。若 Quest 相对完整 attention 已经是近似，HiSparse 不会让它重新等于 full attention。相反，它承诺不再因为缓存驱逐丢失所选的记录。

2 容量墙：少读并不等于少存

设有 N 个并发请求、L 个历史 token、n_ℓ 个需要缓存的层，每个 KV 记录有 w 个元素，每元素 s 字节。完整驻留的主要 KV 开销为

$$M_{\text{full}} \approx NLn_\ell ws.$$

稀疏 attention 每次选择 $k \ll L$ 个位置，计算和读取可以接近 k 的量级，但上式仍由 L 决定。论文举例：一个 128K 的 GLM-5.1 请求，BF16 KV 就占约 13.09 GB；模型权重还需要另外存储。[S1，§1]

HiSparse 为每个 request-layer 预留 B 个 GPU 槽位：

$$M_{\text{hot}} \approx NBn_\ell ws, \qquad B \geq k.$$

当 B 固定，主要 KV payload 不再随该请求历史长度增长。完整历史放到 pinned host DRAM，随时可以重新取回。

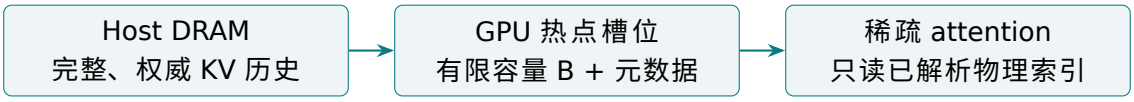
重要限制：整个 GPU 内存不是严格 $O(1)$。选择器的紧凑 indexer state、逻辑到物理的管理信息和其他运行时状态仍然驻留 GPU，其中一部分继续随上下文长度增长。论文明确保留了这些结构，而不是将它们也全部 offload。[S1，§3.2]

2.1 为什么不是只留下当前 top-k

若只分配 $B = k$，下一步选中集合发生变化时，大量刚驱逐的记录可能立刻又被访问。热点缓存的意义是保留一些 “这步没用、下步很可能用” 的条目，而不仅是临时 staging area。

因此容量 B 同时控制两种成本：B 大有利于命中率，却挤压并发数；B 小可接纳更多请求，却增加 host IO。这是典型的容量—带宽联合优化问题，不是简单选择“越大越好”。

3 架构与一个请求的完整生命周期



3.1 Prefill 与 admission

prefill 生成各层 KV 后，将其放入主机 pool；选择器所需的紧凑表示保留在 GPU。进入 decode 时，为请求建立固定大小的缓存与管理结构。若没有足够的 HBM 热点槽位或 host pool，admission 仍必须限制请求，不能将 offload 理解成无穷容量。[S1，§3.3]

3.2 每层 decode 的数据路径

选择器先输出逻辑位置，例如 $\{42, 17, 88, 5\}$。缓存解析器判断哪些记录已命中，哪些缺失；为缺失记录分配可驱逐槽位，从 host 读回，最后返回与选择顺序对齐的物理位置。attention 随后仍在 GPU 上读取这些记录。[S1，Fig.2]

逻辑顺序与物理位置不能混淆。若一个 kernel 把选中 KV 的排列变了，却未同步修正索引和 mask，就可能出现“不崩溃但答案错误”的问题。exact 性能优化首先是一套索引和生命周期正确性协议。

3.3 新生成的 KV：write-through

新 token 的 KV 先写到 GPU 预留位置，使最近内容立即可用；专用备份 stream 把它写回 host。事件依赖必须保证：未来可能重新 fetch 该记录时，host 上已是有效版本。[S1，§3.3]

所以“没有 CPU 同步”不等于没有同步。GPU stream 间仍必须有完成事件、写后读顺序，以及缓存槽位不可提前重用的约束。

4 CUDA Graph 里的动态缓存管理

4.1 固定容量与动态内容不冲突

CUDA Graph 的核心约束是 launch、地址和执行依赖的稳定，不要求每次 token 索引都相同。HiSparse 预分配缓存与工作区，把动态选择、命中和驱逐放在 device 端完成，因此可以复用 graph，而不需要 CPU 每层读取命中数量再决定一组 copy launch。[S1，§3.4]

4.2 RESOLVE 的五个阶段

论文以**每个请求的一个层级 work item 对应一个 CUDA block** 执行融合解析，整体在各 request work item 间并行。[S1，Fig.3]

阶段	输入／工作	为什么需要融合
选中集合入 shared hash	top-k 逻辑位置	后续 membership 查询反复复用
扫描 GPU 槽位	检查 B 个驻留位置	标出 hit 和可驱逐候选
并行 scan 与 LRU 更新	压缩标记、分配 victim	给每个 miss 唯一目标槽位
GPU 发起 host 读取	取回缺失 KV	避免每层 CPU 调度碎片化 DMA
发布 page table 与索引	保持 top-k 顺序	attention 获得完整有效的物理视图

实现使用向 pinned host memory 发起的向量化非一致性 load，例如 `ld.global.nc.v2.b64`，以处理分散的 KV miss。[S1，§3.4] 这不是说 CPU 内存和 HBM 一样快，而是让 GPU 能直接驱动访问，并减少小传输事务与 launch 开销。

4.3 缓存策略有一个特别的细节

命中的旧条目会被提升到刚取回的新条目之前。这个变体偏爱跨步重复出现的 KV：一次性读入的记录，不应立即挤走已经表现出稳定复用的条目。[S1，§3.2]

例如容量为 6，当前已有 A、B、C、D、E、F；本轮选择 A、B、G、H。A/B 命中，G/H 替换两个旧条目；更新后 A/B 的保留优先级高于 G/H。下一轮若 G/H 不再出现，就不必因为刚被取回而长期占据最高优先级。

这只是教学例子。实际并行 LRU 的元数据布局、顺序稳定性和 edge cases 要按实现验证，不能仅凭列表模拟证明 CUDA kernel 无竞争。

5 跨层预取：可证明的复用比猜测更有价值

不同层的 KV 数值不同，不能把上一层的 KV 直接用于下一层。可共享的是**选中位置**。某些模型多个层使用同一组选择索引，HiSparse 利用这一结构，为后续层提前复制它们自己的 KV。[S1，§3.5]

当多个共享层的缓存槽位布局与 anchor 层保持同步时，可以复用 anchor 的搬运计划；后续层等待自己的预取事件，而不用重新执行完整的 probing 和 replacement。这省的是管理工作，也让 IO 被前面的计算覆盖。

“相邻层通常选择相似位置” 则只是预测。论文还测试了 speculative prefetch，但收益很有限：LRU 已经捕捉了大部分容易预测的复用，剩下的 miss 往往恰好难预测。错误 hint 不损害语义，因为最终仍解析真实需求，但会浪费带宽。[S1，§3.5、§4.6]

这是很有价值的负面结果：**更多预取不一定更快，尤其当共享 host 带宽已经接近饱和时**。

6 实验结果：收益主要来自更大的可服务并发

6.1 硬件与精度先说明

论文在 SGLang v0.5.11 上与完整 KV 驻留基线比较，覆盖 DSA、NSA 和 Quest。不同测试使用 B200、H200、GH200；它们的 host-device 互联并不相同。权重标为 FP8 不代表 KV 也是 FP8，论文的这些 KV 实验使用 BF16。[S1，§4.1]

模型与场景	基线峰值生成吞吐	HiSparse	比值
Qwen3-30B-A3B，32K 输入	511 tok/s	1,824 tok/s	3.6×
Qwen3-30B-A3B，200K 输入	111 tok/s	520 tok/s	4.7×
GLM-5.1，32K 输入	624 tok/s	1,919 tok/s	3.1×
GLM-5.1，160K 输入	232 tok/s	680 tok/s	2.9×

这是论文 §4.1–4.2 报告的工作负载结果，不是同一个小 batch 下单 attention kernel 的速度比。不同模型使用不同平台，不能把横向行间数值当作模型效率排行榜。

在短上下文，优势会减小甚至消失：论文给出的 4K GLM 结果约 2,288→2,280 tok/s，说明当完整 KV 本来就不构成容量压力时，额外管理和 IO 不一定值得。[S1，§4.2]

6.2 TTFT 的收益为何也可能很大

当 decode 请求占满 HBM，prefill 只能排队，或系统必须缩小活跃请求集合。HiSparse 降低 decode 驻留，使 prefill 更容易获得资源。因此 TTFT 的改善不一定来自 prefill GEMM 变快，而可能来自排队时间和 admission 的改变。

报告在 DeepSeek-V4-Flash 的高并发示例中给出明显 TTFT 降低，同时低并发下 TPOT 近似持平。[S1，§4.1] 这种 “容量改变排队” 的系统收益，应与执行时间优化分开计量。

论文还使用 decode-only 吞吐近似分离式部署的 decode 阶段，但并未把完整物理分离部署的网络与控制开销全部测出来。因此不能把 decode-only 数值直接当作完整 PD 分离系统的端到端 SLA。

7 缓存命中率实验究竟说明什么

作者用同一 GLM-5.1 选择轨迹比较策略：prompt 100,384 token、生成 1,799 步，跨 78 个稀疏层；主要统计覆盖前 1,000 decode steps。[S1，§4.3、Fig.6]

策略	每 request-layer 槽位 B	平均 miss rate
仅当前 top-k staging	2,048	30.0%
FIFO	4,096	17.2%
Random	4,096	16.1%
LRU	4,096	13.4%
离线 Bélády oracle	4,096	8.2%
LRU	8,192	6.7%

Bélády 利用未来访问，是性能上界参照，不是可以无代价在线部署的算法。LRU 从 13.4% 降到 6.7% 需要加倍缓存；并发容量因此下降。所以不能仅凭 miss rate 最低，就判定该配置服务吞吐最好。

例如，假设带宽压力能随 miss rate 近似减半，但可容纳请求也减半，最终 aggregate throughput 未必增加。这是解释性的反例，不是对论文测量的否定。

8 一个可用于复现的成本模型

下面是本报告的分析模型。对 N 个请求、一轮 decode，跨层需要搬运的主要字节量近似为

$$D_{\text{host}} \approx \sum_{r=1}^{N} \sum_{\ell} k_{\ell}(1 - h_{r\ell}) w_{\ell} s.$$

理想传输时间下界为 $D_{\text{host}}/BW_{\text{effective}}$，但实际还受碎片化、NUMA、并发 load、PCIe 或 C2C 事务效率影响。

$$T_{\text{step}} \approx T_{\text{model}} + T_{\text{resolve}} + T_{\text{IO,exposed}}.$$

预取只隐藏有独立计算覆盖的部分，不能把所有 IO 都从时间线上消掉。B 增大时，h 可能提高，resolve 的扫描成本却上升，同时最大并发

$$N_{\max} \lesssim \frac{M_{\text{HBM,available}}}{B n_{\ell} w s + M_{\text{other/request}}}$$

会下降。优化目标应是满足延迟约束的有效吞吐，而不是单独最大化命中率。

这个模型也解释了为什么 GH200 上的结果不能直接移植到普通双路 CPU 加 PCIe 的 B200 服务器：计算卡看起来都很强，host 路径却可能完全不同。

9 exact 语义的验收要比性能更先做

应固定 query、选择索引和 KV 内容，分别用全 HBM、全 miss、混合 hit/miss、重复位置、跨 chunk 边界和长时间回放测试。检查最终读取记录、顺序、attention 输出及 logits，而不只检查输出文本是否碰巧一样。

重点故障点包括：write-through 尚未完成就 fetch；错误复用刚释放的请求槽；LRU victim 仍被 attention 使用；跨层共享索引但拿错层 KV；请求 fork 后共享可写状态；graph replay 引用了旧地址。

缓存 exact 不自动推出跨不同 attention kernel 的 bitwise 等价。若取回后的计算顺序与 baseline 一致，可以追求更强的一致性；若采用不同 reduction 布局，则应区分 “数据没有损失” 与 “浮点计算顺序相同”。

10 局限、部署代价与研究空间

首先，系统把压力转移到主机内存和总线，并没有消灭历史存储。大量长请求需要很大的 pinned pool；它可能影响操作系统可分页内存、NUMA 分配和其他作业。论文最大的并发配置使用了很大的 host memory，不能按 “只要几 GB HBM” 理解整机需求。[S1，§4.1]

其次，管理层支持多种 selector，不等于任意 attention 架构都可以零改动接入。索引粒度、重复索引、compressed KV、page 大小和 layer sharing 都需要适配。

最后，报告以完整 HBM 驻留为主要基线，并非证明优于所有 offload/cache 系统。no-IO oracle 显示管理成本在其设置下没有可测的端到端影响，也不能推广成所有 batch、所有卡上 “解析开销等于零”。

最值得继续研究的方向是联合 admission 与 B 的选择、利用真正共享的层索引，以及带 NUMA/总线约束的热点工作集调度。新的替换算法必须把额外预测与管理成本计入，而不是只报告比 LRU 少几个百分点的 miss。

11 复现顺序与资料

第一阶段只重放固定选择轨迹，验证缓存协议；第二阶段在实际 host-device 拓扑上测分散读取与 RESOLVE；第三阶段才接入模型，并逐步扫描请求并发、上下文长度和 B。GPU event 计时、wall time 与真实输出 token 计数应同时保留。

服务测量要把 prefill、decode、排队与数据搬运分开。固定相同请求集，既比较等并发延迟，也比较等 SLA 下最大吞吐，避免仅因为采用不同 admission 规则就得出没有可比性的加速比。

[S1] 原论文。HiSparse, arXiv:2608.07009v1。重点阅读 §3.2–3.5、Fig.2–3、§4 与 Fig.6。性能与缓存数据均为作者报告。

[S2] 框架入口。 SGLang 官方仓库。论文报告其集成于上游；复现时仍需锁定论文所用版本及相应实现，不能默认当前分支与论文设置完全一致。

本文没有运行 SGLang、缓存 CUDA kernel 或大模型吞吐测试。容量模型、教学例子与验收矩阵属于解释性分析。